

What is NumPy?

NumPy is a powerful Python library used for:

- Fast mathematical calculations
- Working with arrays and matrices
- Data analysis
- Machine Learning
- Scientific computing
- Image processing
- AI and Deep Learning

NumPy stands for:

Numerical Python

Why Do We Need NumPy?

Before NumPy, Python used normal lists.

Example:

```
marks = [45, 67, 89, 90]
```

Python lists are:

- Slow for large calculations
- Use more memory
- Difficult for advanced mathematical operations

NumPy solves these problems.

Real-Time Analogy

Imagine you work in a company and have:

- 10 lakh sales records
- Employee attendance data
- Sensor readings every second
- Hospital patient records

Using normal Python lists becomes slow.

NumPy processes huge data very fast because it is optimized in C internally.

Example Without NumPy

```
a = [1, 2, 3]
b = [4, 5, 6]

result = []

for i in range(len(a)):
    result.append(a[i] + b[i])

print(result)
```

Output:

[5, 7, 9]

Example With NumPy

```
import numpy as np

a = np.array([1, 2, 3])
b = np.array([4, 5, 6])

print(a + b)
```

Output:

[5 7 9]

Much shorter and much faster.

Main Advantage of NumPy

Feature	Python List NumPy	
Speed	Slow	Very Fast
Memory	High	Less
Mathematical Operations	Difficult	Easy
Multi-dimensional Data	Limited	Excellent
Used in AI/Data Science	Rarely	Everywhere

What is an Array?

An array is like a container that stores multiple values of the same type.

Example:

```
import numpy as np
```

```
arr = np.array([10, 20, 30, 40])  
print(arr)
```

Output:

```
[10 20 30 40]
```

What is 1D Array?

1D means **one dimension**.

It is like a single row.

```
arr = np.array([1, 2, 3, 4])
```

Visualization:

```
[1 2 3 4]
```

Real-time example:

- Student marks
 - Monthly sales
 - Temperature readings
-

What is 2D Array?

2D means **two dimensions**.

It has:

- Rows
- Columns

Like an Excel sheet.

```
arr = np.array([  
    [1, 2, 3],  
    [4, 5, 6]  
])
```

```
print(arr)
```

Visualization:

```
[  
 [1 2 3]  
 [4 5 6]  
]
```

Real-time examples:

- Excel tables
- Employee salary sheet
- School mark list
- Database tables

Why 2D is Important?

Suppose you are a Data Analyst.

Sales data:

Product Jan Feb Mar

Mobile 100 120 130

Laptop 80 90 110

This is naturally 2D data:

- Rows → Products
- Columns → Months

NumPy handles this efficiently.

What is 3D Array?

3D means multiple 2D arrays stacked together.

Think of it like:

- Multiple Excel sheets
- Video frames
- MRI scan layers

Example:

```
arr = np.array([
    [
        [1, 2],
        [3, 4]
    ],
    [
        [5, 6],
        [7, 8]
    ]
])
```

```
])
```

```
print(arr)
```

Visualization:

Sheet 1

```
[  
[1 2]  
[3 4]  
]
```

Sheet 2

```
[  
[5 6]  
[7 8]  
]
```

Real-Time Examples of 3D Arrays

1. Image Processing

A color image has:

- Height
- Width
- RGB colors

Example:

- $1920 \times 1080 \times 3$

3 dimensions:

1. Height
2. Width
3. Color channel

Used in:

- Face recognition
 - Photoshop filters
 - Medical imaging
-

2. Video Processing

Video is:

- Collection of images (frames)

Dimensions:

- Frame number
- Height
- Width
- Colors

Actually videos often become 4D arrays.

3. Hospital MRI Scan

Doctors scan body layers.

Each layer = 2D image

Many layers together = 3D data

Higher Dimensions (4D, 5D...)

Used in:

- Deep Learning
- AI models
- Scientific simulations
- Weather forecasting

Example in AI:

(batch_size, height, width, channels)

Important NumPy Concepts

1. Shape

Shape tells dimensions.

`arr.shape`

Example:

```
arr = np.array([
    [1,2,3],
    [4,5,6]
])
```

```
print(arr.shape)
```

Output:

(2, 3)

Meaning:

- 2 rows
 - 3 columns
-

2. Dimension

`arr.ndim`

Example:

```
print(arr.ndim)
```

Output:

2

Means 2D array.

3. Data Type

`arr.dtype`

Example:

```
arr = np.array([1,2,3])
```

```
print(arr.dtype)
```

Output:

int64

4. Indexing

Access elements.

```
arr = np.array([10,20,30])
```

```
print(arr[1])
```

Output:

20

5. Slicing

```
print(arr[0:2])
```

Output:

```
[10 20]
```

Mathematical Operations

```
a = np.array([1,2,3])
```

```
print(a + 10)
```

```
print(a * 2)
```

```
print(a ** 2)
```

Output:

```
[11 12 13]
```

```
[2 4 6]
```

```
[1 4 9]
```

Statistical Operations

```
arr = np.array([10,20,30,40])
```

```
print(np.mean(arr))
```

```
print(np.sum(arr))
```

```
print(np.max(arr))
```

```
print(np.min(arr))
```

Output:

```
25.0
```

```
100
```

```
40
```

```
10
```

Real-Time Data Analyst Example

Suppose you have company sales data.

```
sales = np.array([100, 120, 140, 160, 180])
```

```
print("Total Sales:", np.sum(sales))
```

```
print("Average Sales:", np.mean(sales))
```

```
print("Highest Sales:", np.max(sales))
```

Output:

Total Sales: 700
Average Sales: 140.0
Highest Sales: 180

This is why NumPy is heavily used in:

- Data Analytics
- Power BI backend processing
- Machine Learning
- Forecasting

NumPy in Data Science Ecosystem

Many popular libraries depend on NumPy:

Library	Purpose
Pandas	Data Analysis
Matplotlib	Charts
Scikit-learn	Machine Learning
TensorFlow	Deep Learning
OpenCV	Image Processing

Without NumPy, most modern Python data science tools would not work efficiently.

Where Companies Use NumPy

Banking

- Fraud detection
- Risk analysis

Healthcare

- MRI scan analysis
- Patient data analysis

E-commerce

- Recommendation systems

IT Companies

- Dashboard analytics
- KPI calculations

Manufacturing

- Sensor data processing

AI Applications

- Face recognition
 - Chatbots
 - Self-driving cars
-

Simple Memory Trick

Dimension Think Like		Example
1D	Line	Student marks
2D	Table	Excel sheet
3D	Multiple tables	Image/video layers
4D+	Collection over time AI data	